

IS311 Web Development

Week 9 - Lecture 10

Dr. Abbas Malik
amaalik@psu.edu.sa

```
const http = require('http');
const Adder = require('../events/adder');
const adder = new Adder();

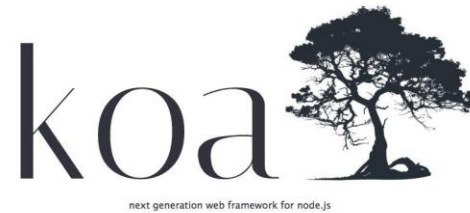
const serverObj = http.createServer(
  (request, response) => { // callback function
    if(request.url === '/') {
      response.write('Hello World from Node');
      response.end();
    }
    if(request.url === '/add') {
      adder.on('addedEvent', function() {
        response.write(`Sum of numbers is ${adder.sum}`);
        response.end();
      });
      adder.add(34, 56);
    }
  }
);
serverObj.listen(3000);
console.log('Listening on port 3000...');
```

Web Apps

- Can be developed using Node.js only
- But a framework make it easier to maintain, manage and code a web application using Node.js

Frameworks

- Web apps can be quickly and efficiently built
- Streamline development activities



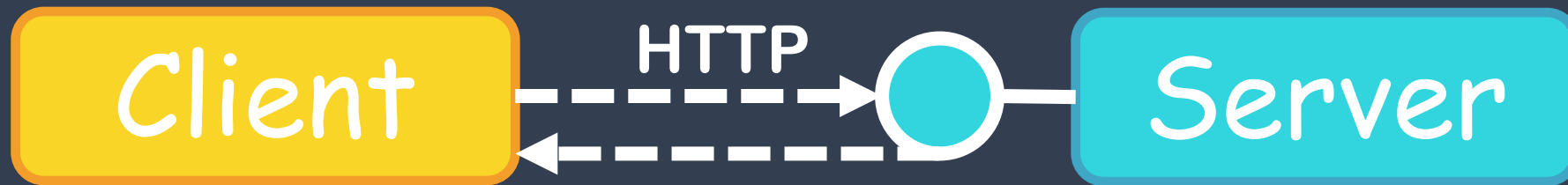


- Lightweight and adaptable
- Easy development
- Well-documented
- Generate HTML pages dynamically
- MVC - Model-View-Controller Design pattern
- Event-driven nature (callbacks)

HyperText Transfer Protocol

- Request and response stateless protocol
 1. Client opens connection to server
 2. Client sends HTTP request to server
 3. Server receives and process request
 4. Server sends back the response
 5. Connection to server is closed

Client - Server Architecture



Representational **S**tate **T**ransfer REST

CRUD Operations

Create

Read

Update

Delete

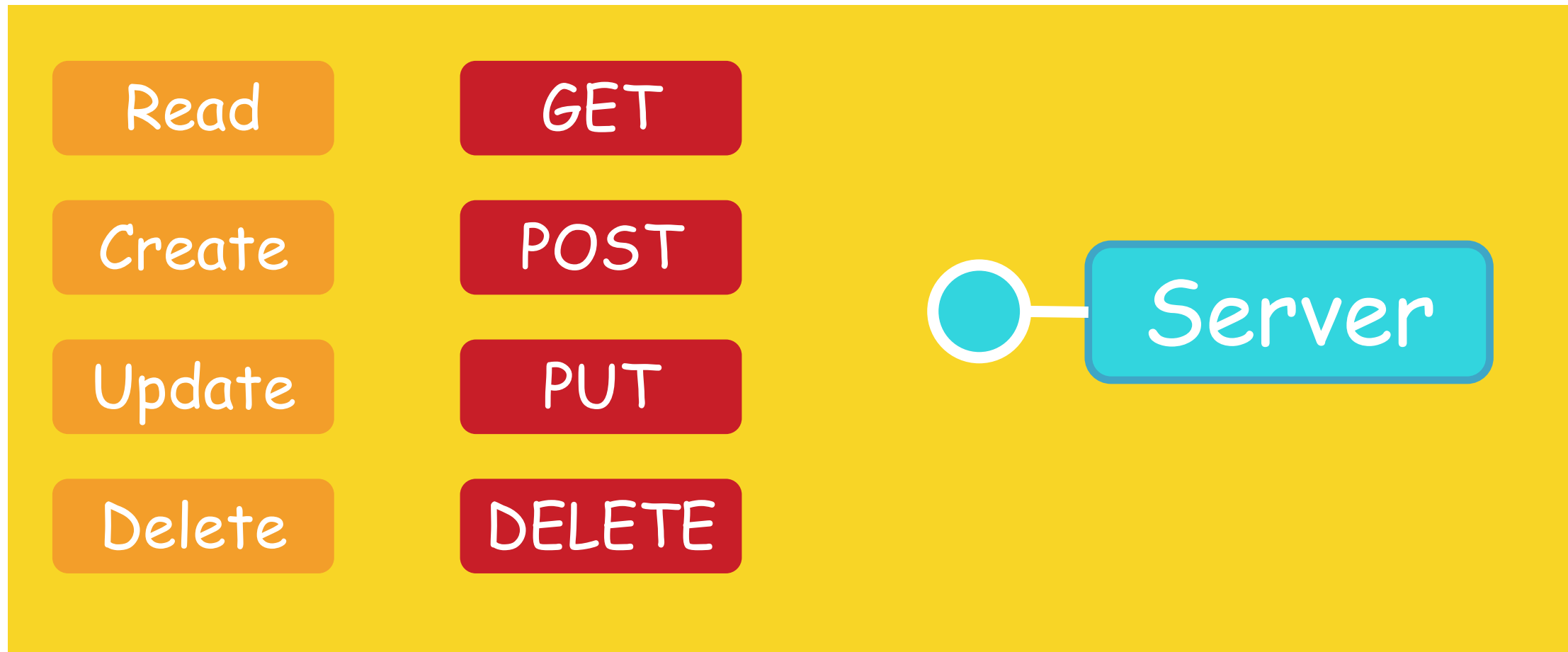


Server

Manage Students at PSU-CCIS

<http://www.psu.edu/ccis/data/students>

HTTP Methods



Request and Response - Read Data

Request

GET /data/students

Response

```
[  
  {"id": 1, "name": "Abbas"},  
  {"id": 2, "name": "John"},  
  ...  
]
```

Request and Response - Read Data

Request

GET /data/students/1

Response

```
{“id”: 1, “name”: “Abbas”}
```

Request and Response - Update Data

Request

```
PUT /data/students/1  
{  
  "name": "M G Abbas"  
}
```

Response

```
{  
  "id": 1,  
  "name": "M G Abbas"  
}
```

Request and Response - Delete Data

Request

DELETE /data/students/1

Response

```
{  
  "result": "Success"  
}
```

Request and Response - Add Data

Request

```
POST /data/students/  
{  
  "name": "Moeez"  
}
```

Response

```
{  
  "id": 3,  
  "name": "Moeez"  
}
```


Request and Response - HTTP Request

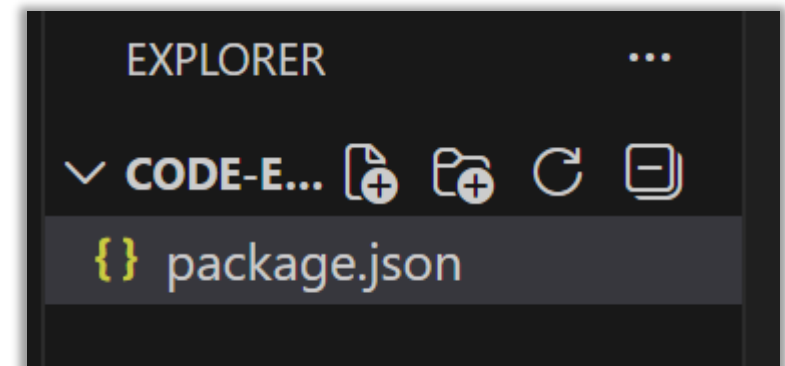
HTTP Request

GET /data/students
GET /data/students/1
POST /data/students/
PUT /data/students/2
DELETE /data/students/3



Web Service to Manage Students

- Create a folder 'code-examples' and open in VS Code
- Open terminal and run the command
`npm init -y`
- It will create a package.json file in this folder



package.json

```
$ npm init -y
Wrote to D:\Teaching\IS311 Web Development\Lectures\Week 09\Code\code-examples\package.json:

{
  "name": "code-examples",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```

EXPLORER  ...
  CODE-EXAMPLES
    {} package.json

{} package.json X
{} package.json > {} scripts
1  {
2    "name": "code-examples",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
   ▶ Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs"
13  }

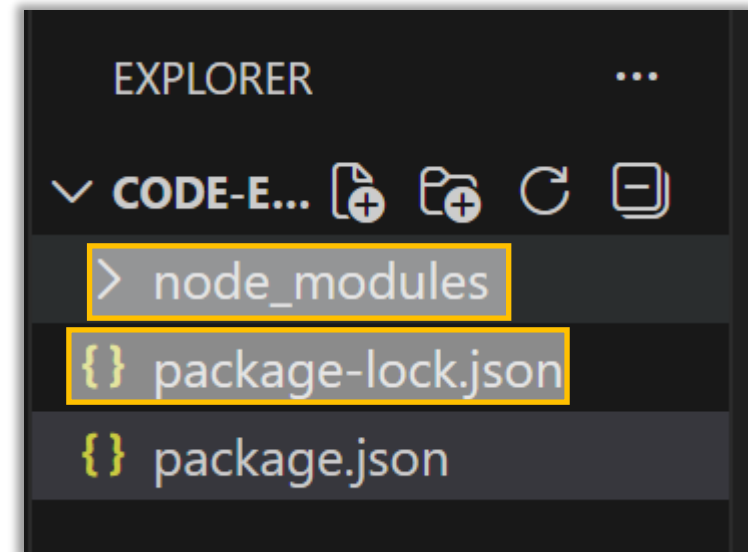
```

Installing Express

- In terminal, run the following command

`npm i express`

- This will add a few new files in this folder

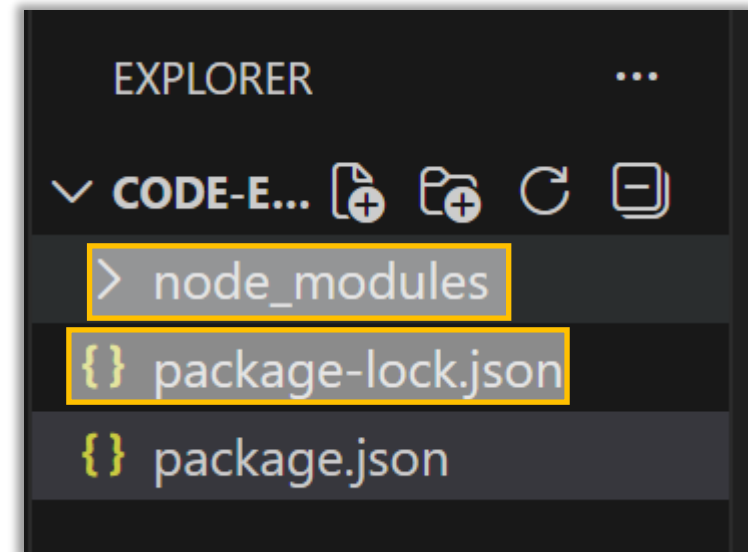


Installing nodemon

- In terminal, run the following command

`npm i -D nodemon`

- This will add a few new files in this folder



package.json

Initial file

```
{ package.json
{} package.json > ...
1 {
2   "name": "code-examples",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   > Debug
7   "scripts": {
8     "test": "echo \"Error: no test specified\" && exit 1"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs"
}
```

```
{ package.json X
{} package.json > {} scripts > abc dev
1 {
2   "name": "code-examples",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   > Debug
7   "scripts": {
8     "dev": "nodemon index.js"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs",
14  "dependencies": {
15    "express": "^5.2.1"
16  },
17  "devDependencies": {
18    "nodemon": "^3.1.11"
19  }
}
```

Express

- API Documentation
 - <https://expressjs.com/en/4x/api.html>
- Application
- Request
- Response
- Router

index.js

- In VS Code, create a new file and name it index.js
- Import express module and edit the file

index.js

```
const express = require("express");
```

```
const app = express();
```

`express()`: Express

Creates an Express application. The `express()` function is a top-level function exported by the `express` module.

index.js

```
const express = require("express");  
const app = express();
```

```
app.get("URL", function (req, res) {});  
app.post("URL", function (req, res) {});  
app.put("URL", function (req, res) {});  
app.delete("URL", function (req, res) {});
```

These functions can take two arguments:

- (1) String route
- (2) Callback function

index.js

```
const express = require("express");  
const app = express();  
console.log("Express App is created...");
```

```
app.get("/", (req, res) => {  
  res.send("Hello World from Node JS Server");  
});
```

Implemented a route "/" to root that will return the 'Hello World!!!' message to the browser

index.js

```
const express = require("express");  
const app = express();  
console.log("Express App is created...");  
  
app.get("/", (req, res) => {  
  res.send("Hello World from Node JS Server");  
});
```

```
app.listen(3000, () =>  
  console.log("Express App is waiting on port 3000 for any HTTP  
request!!!")  
);
```

index.js

The image shows a VS Code editor window with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'CODE-EXAMPLES' with a subdirectory 'node_modules' and files 'package-lock.json' and 'package.json'. The code editor shows the content of 'index.js'.

```

1  const express = require("express");
2  const app = express();
3
4  console.log("Express App is created...");
5
6  app.get("/", (req, res) => {
7    res.send("Hello World from Node JS Server");
8  });
9
10 app.listen(3000, () =>
11   console.log("Express App is waiting on port 3000 for any HTTP request!!!")
12 );
13

```

Below the code editor is a terminal window showing the command to run the application and its output:

```

malik@DESKTOP-HVT4456 MINGW64 /d/Teaching/IS311 Web Development/Lectures/Week 09/Code/code-examples
$ node index.js
Express App is created...
Express App is waiting on port 3000 for any HTTP request!!!

```

Overlaid on the right side of the code editor is a browser window showing the output of the application: "Hello World from Node JS Server". A red arrow points from the text "Hello World from Node JS Server" in the browser to the corresponding line in the code editor.

node index.js

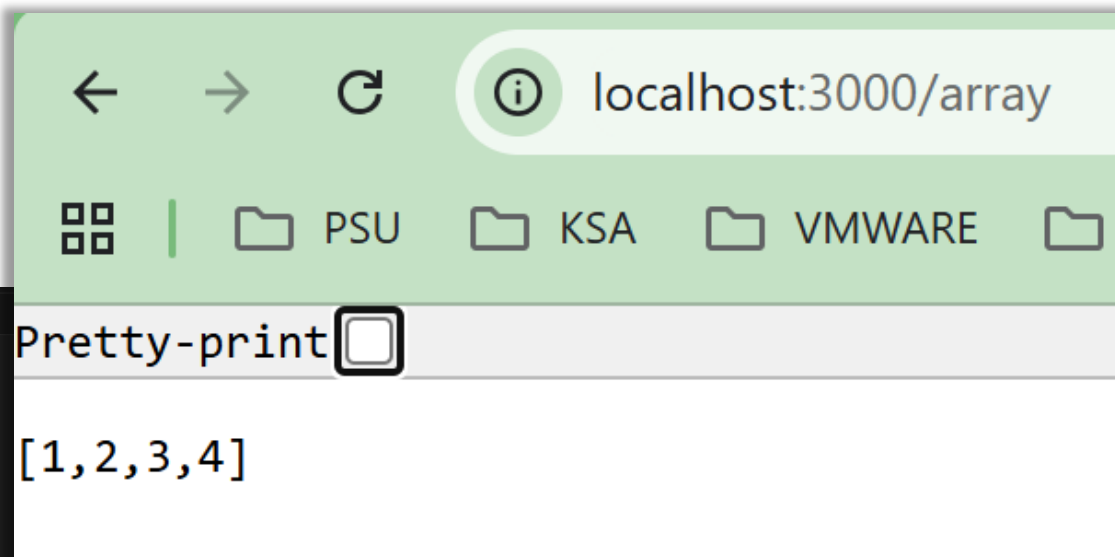
index.js

- Add another route in this file with *GET* method

```
app.get("/array", function (req, res) {  
  res.send([1, 2, 3, 4]);  
});
```

Stope the previous execution by pressing "ctrl + c" and run again the newly modified program after saving index.js file

index.js



```

EXPLORER
  CODE-EXAMPLES
    > node_modules
    JS index.js
    package-lock.json
    package.json
  JS index.js > ...
1  const express = require("express");
2  const app = express();
3
4  console.log("Express App is created...");
5
6  app.get("/", (req, res) => {
7    res.send("<h1>Hello World from Node JS Server</h1>");
8  });
9
10 app.get("/array", function (req, res) {
11   res.send([1, 2, 3, 4]);
12 });
13
14 app.listen(3000, () =>
15   console.log("Express App is waiting on port 3000 for any HTTP request!!!")
16 );

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
$ npm run dev
> code-examples@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Express App is created...
Express App is waiting on port 3000 for any HTTP request!!!
  
```

npm run dev

nodemon Package

- To get rid of running program again and again, we can install nodemon package using NPM
- To install run the follownig command
`npm i -D nodemon`
- Edit package.json file and run the server with nodemon

Dynamic Web Development

- Data is coming from different sources
 - Database
 - Cloud
 - Files
 - Streams
 - ...
- Based on data, web pages are changing - making them dynamic

MySQL DBMS

- To install MySQL
 - <https://www.apachefriends.org/index.html>



XAMPP Apache + MariaDB + PHP + Perl

What is XAMPP?

XAMPP is the most popular PHP development environment

XAMPP is a completely free, easy to install Apache distribution containing MariaDB, PHP, and Perl. The XAMPP open source package has been set up to be incredibly easy to install and to use.



XAMPP

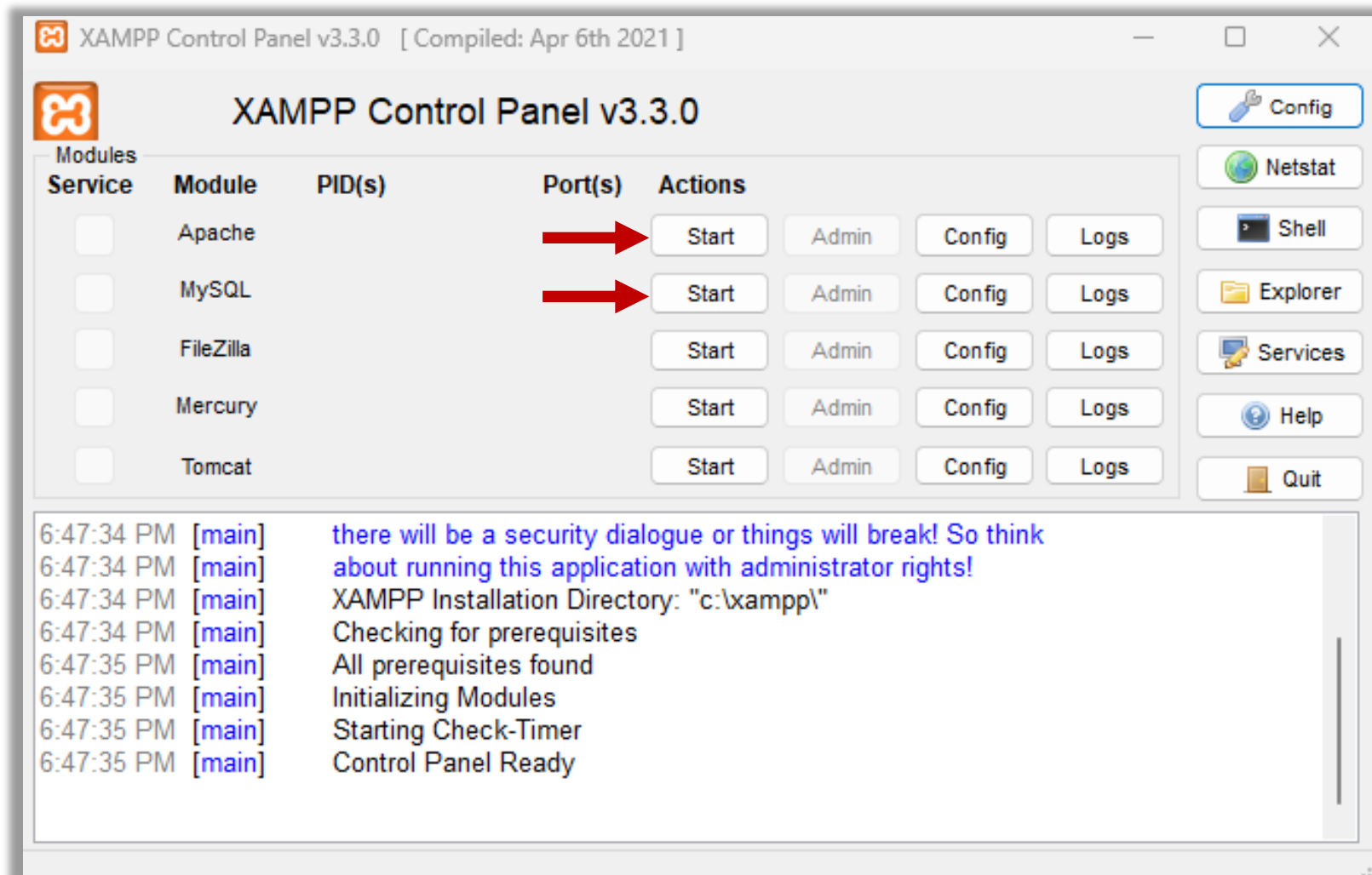
Download
Click here for other versions

 **XAMPP for Windows**
8.2.12 (PHP 8.2.12)

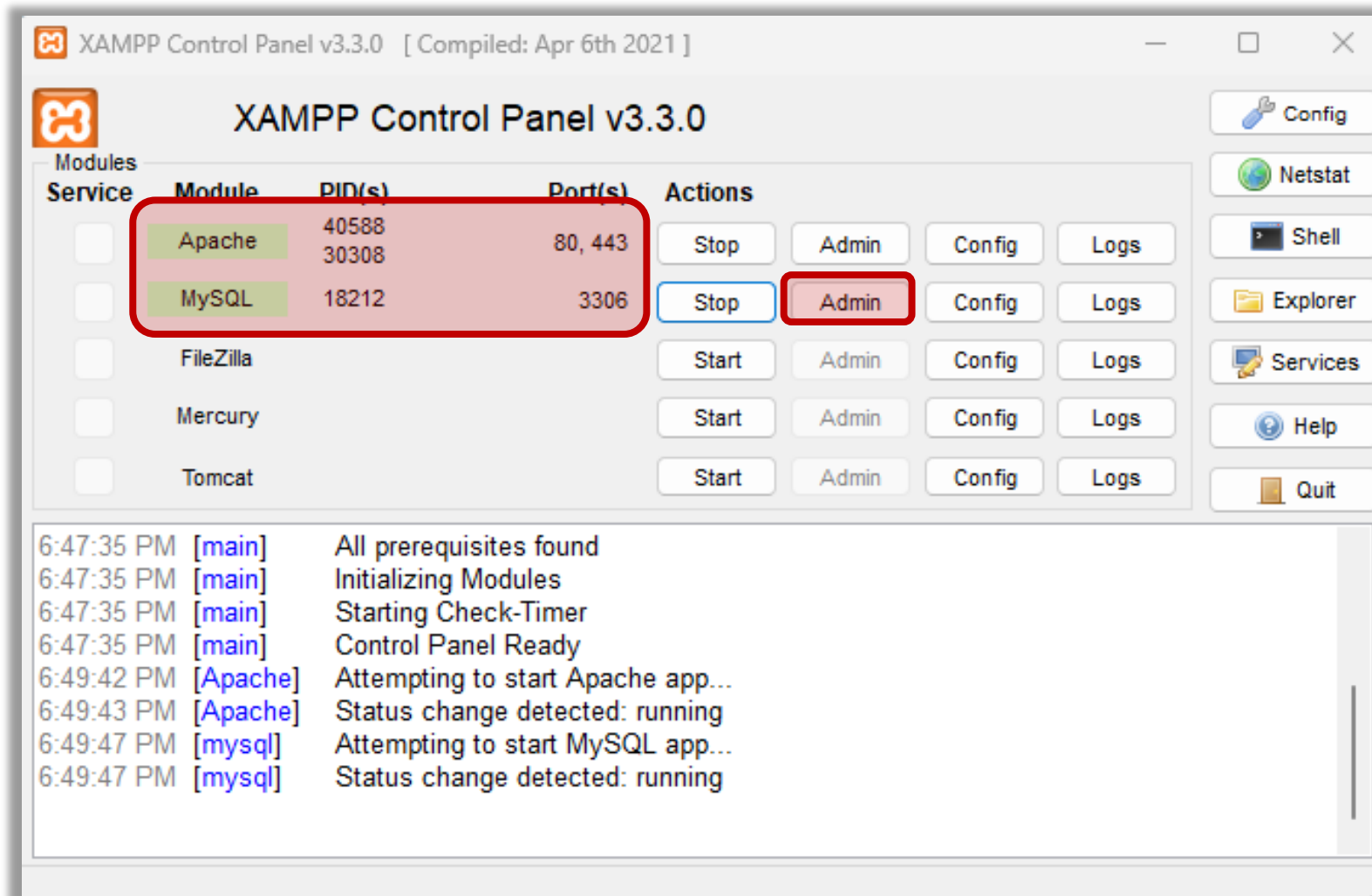
 **XAMPP for Linux**
8.2.12 (PHP 8.2.12)

 **XAMPP for OS X**
8.2.4 (PHP 8.2.4)

XAMPP - Admin



XAMPP - Admin



The screenshot displays the phpMyAdmin web interface for a local MySQL/MariaDB server. The 'Databases' tab is highlighted in the top navigation bar. The left sidebar shows a list of databases: information_schema, mysql, performance_schema, phpmyadmin, and test. The main content area is divided into several panels:

- General settings:** Shows the server connection collation set to 'utf8mb4_unicode_ci' and a link to 'More settings'.
- Appearance settings:** Shows the language set to 'English' and the theme set to 'pmahomme'.
- Database server:** Provides details about the server configuration, including:
 - Server: Localhost via UNIX socket
 - Server type: MariaDB
 - Server connection: SSL is not being used
 - Server version: 10.4.28-MariaDB - Source distribution
 - Protocol version: 10
 - User: root@localhost
 - Server charset: UTF-8 Unicode (utf8mb4)
- Web server:** Provides details about the web server environment, including:
 - Apache/2.4.56 (Unix) OpenSSL/1.1.1t PHP/8.2.4 mod_perl/2.0.12 Perl/v5.34.1
 - Database client version: libmysql - mysqlnd 8.2.4
 - PHP extension: mysqli, curl, mbstring
 - PHP version: 8.2.4
- phpMyAdmin:** Provides information about the application itself, including:
 - Version information: 5.2.1 (up to date)
 - Links to Documentation, Official Homepage, Contribute, Get support, List of changes, and License.

Databases


Create database


▼

☐ Check all

	Database	Collation	Action
☐	information_schema	utf8_general_ci	 Check privileges
☐	mysql	utf8mb4_general_ci	 Check privileges
☐	performance_schema	utf8_general_ci	 Check privileges
☐	phpmyadmin	utf8_bin	 Check privileges
☐	test	utf8mb4_general_ci	 Check privileges

Total: 5










Recent
 Favorites

 New


 information_schema


 mysql


 **nodejsdb**


 performance_schema


 phpmyadmin


 test

Server: localhost » Database: nodejsdb

Structure
 SQL
 Search
 Query
 Export
 Import

 No tables found in database.

 Create new table

Table name

Number of columns

4



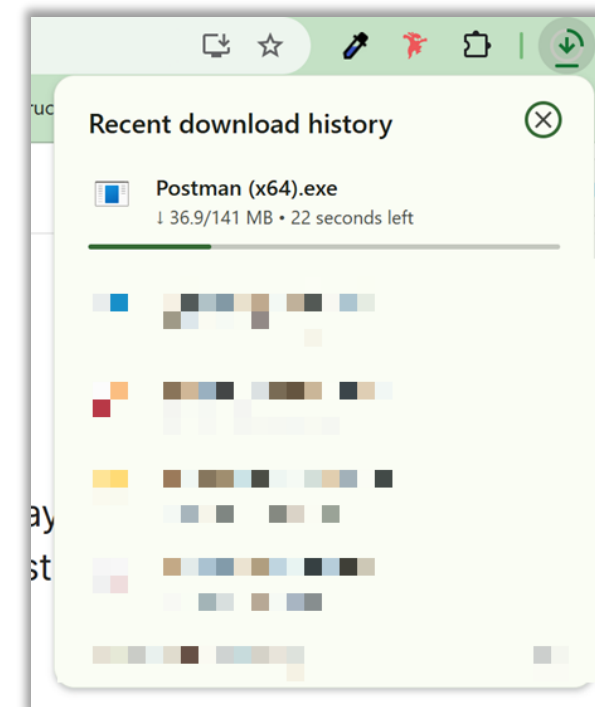
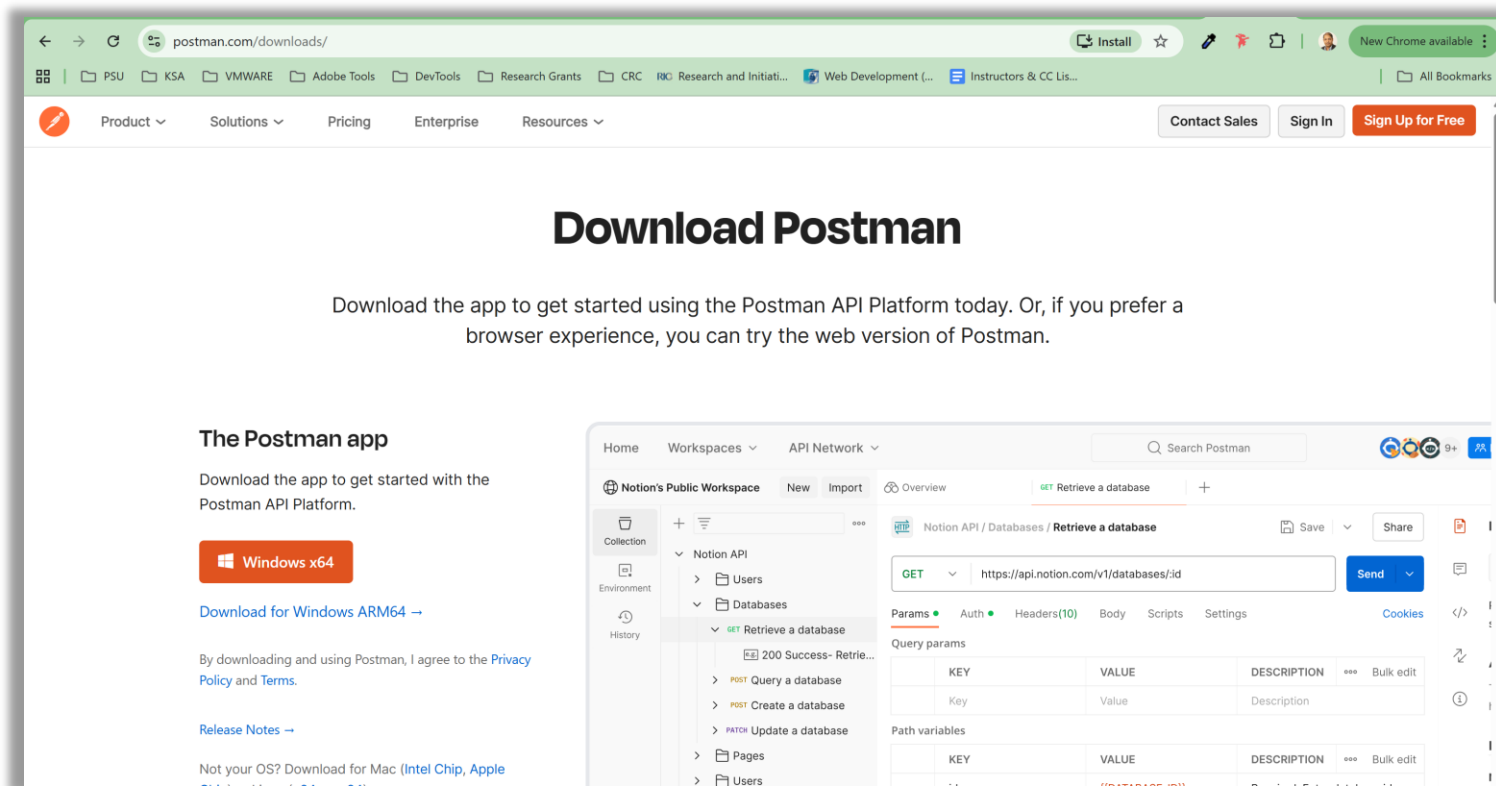

Create

MySQL DBMS

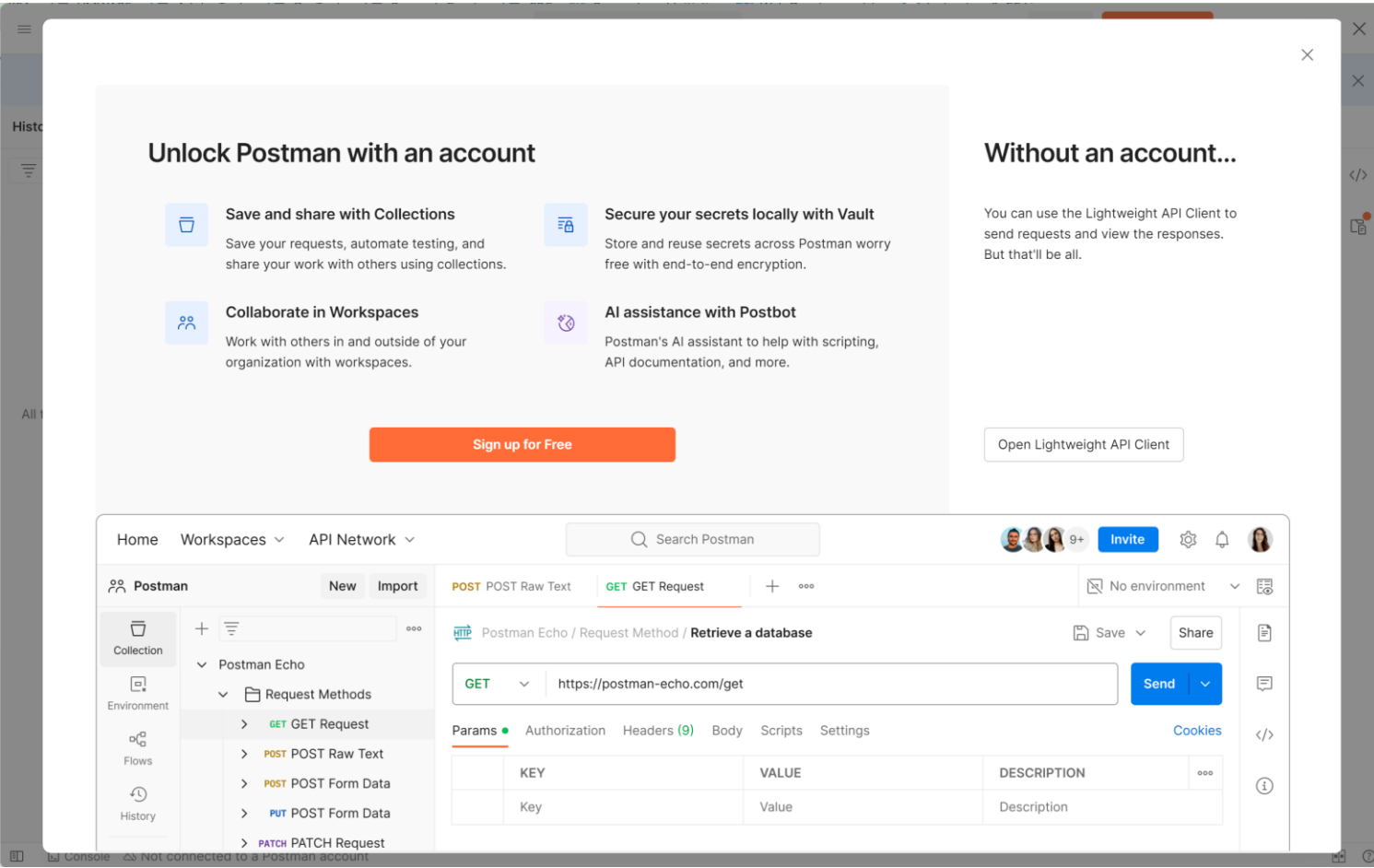
- To learn how to code in SQL in MYSQL
- Please follow the following tutorial
- <https://www.w3schools.com/mysql/default.asp>

Postman Application

- <https://www.postman.com/downloads/>



Postman Application



Postman Application



Why sign up?

- Collaborate with your team for free
- Build, test, and share APIs in one place
- Automate your API workflows
- Accelerate your work with AI



Create Postman account

Work email

Username

Password Show

☐ Receive product updates, news, and other marketing communications

☒ Stay signed in


Success!

By creating an account, you agree to our [terms](#) and [privacy policy](#).

Create Free Account

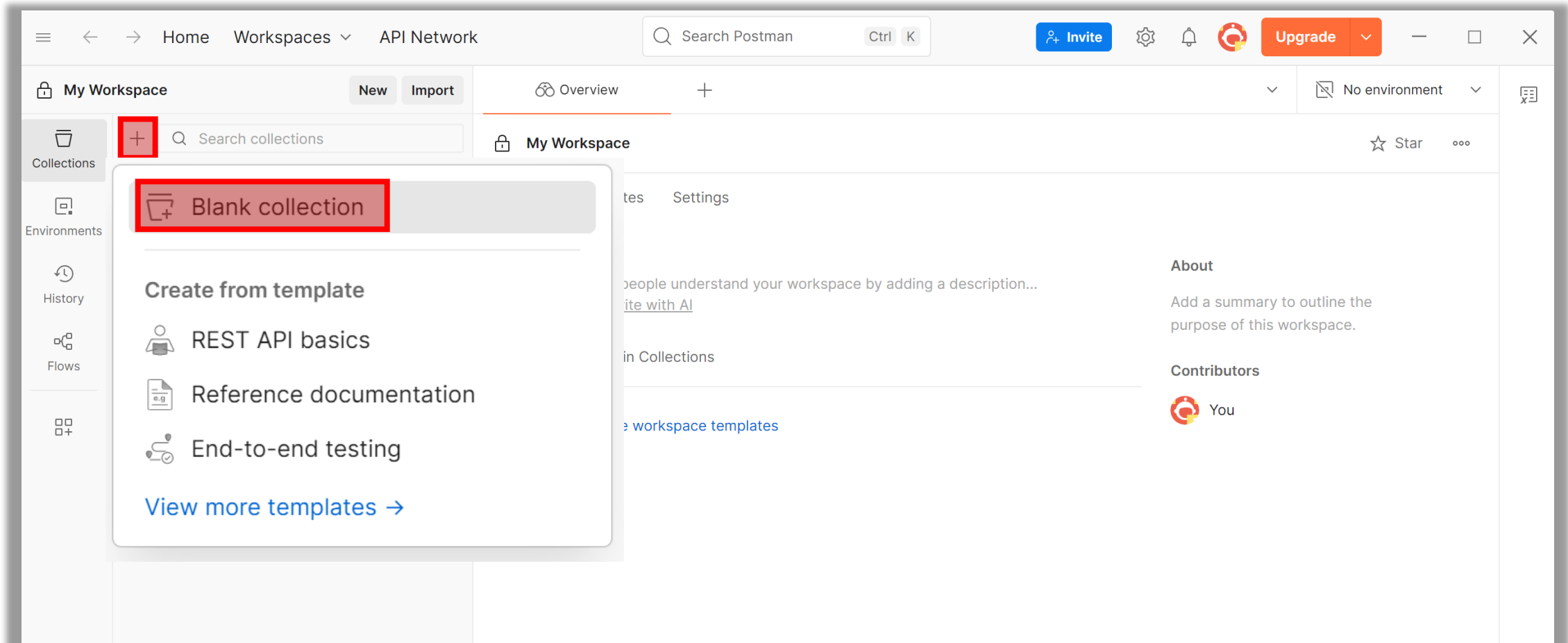
or

 Sign Up with Google

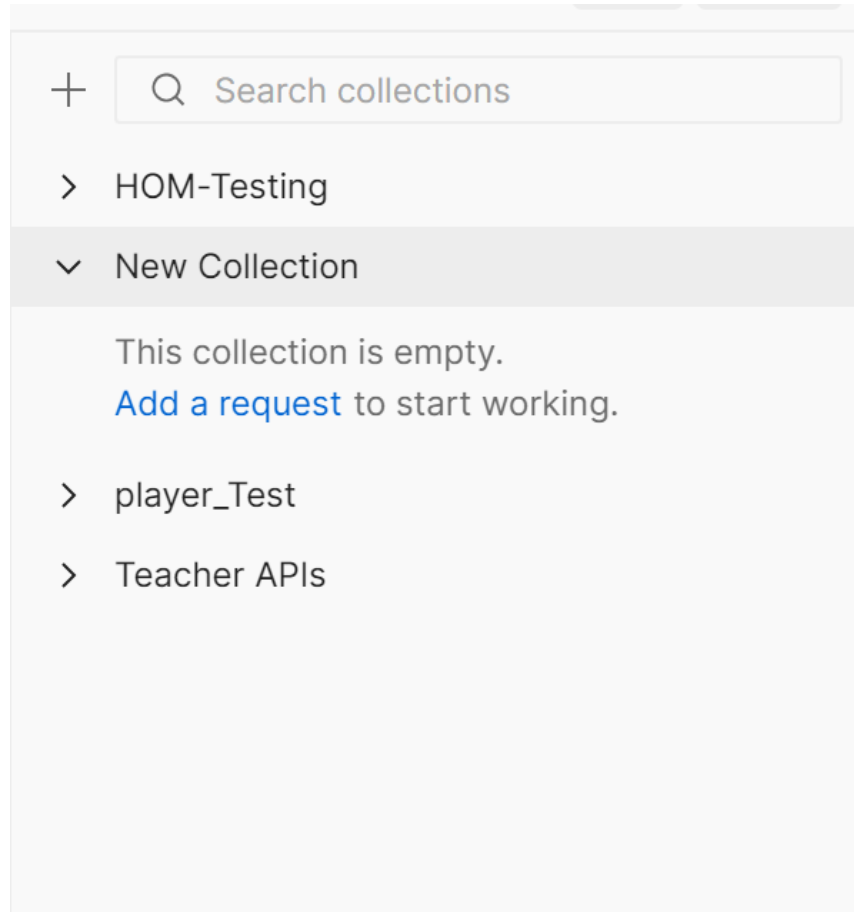
 Sign Up with GitHub

Sign In with SSO

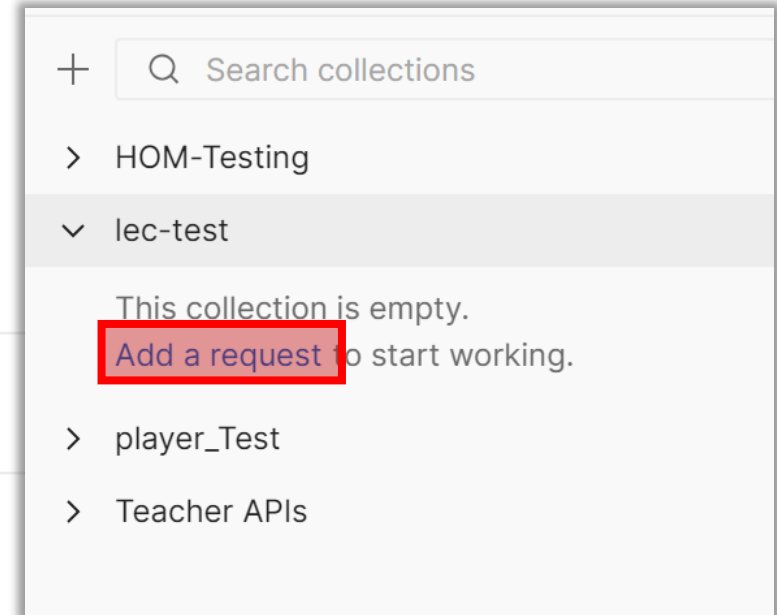
Postman Application



Postman Application



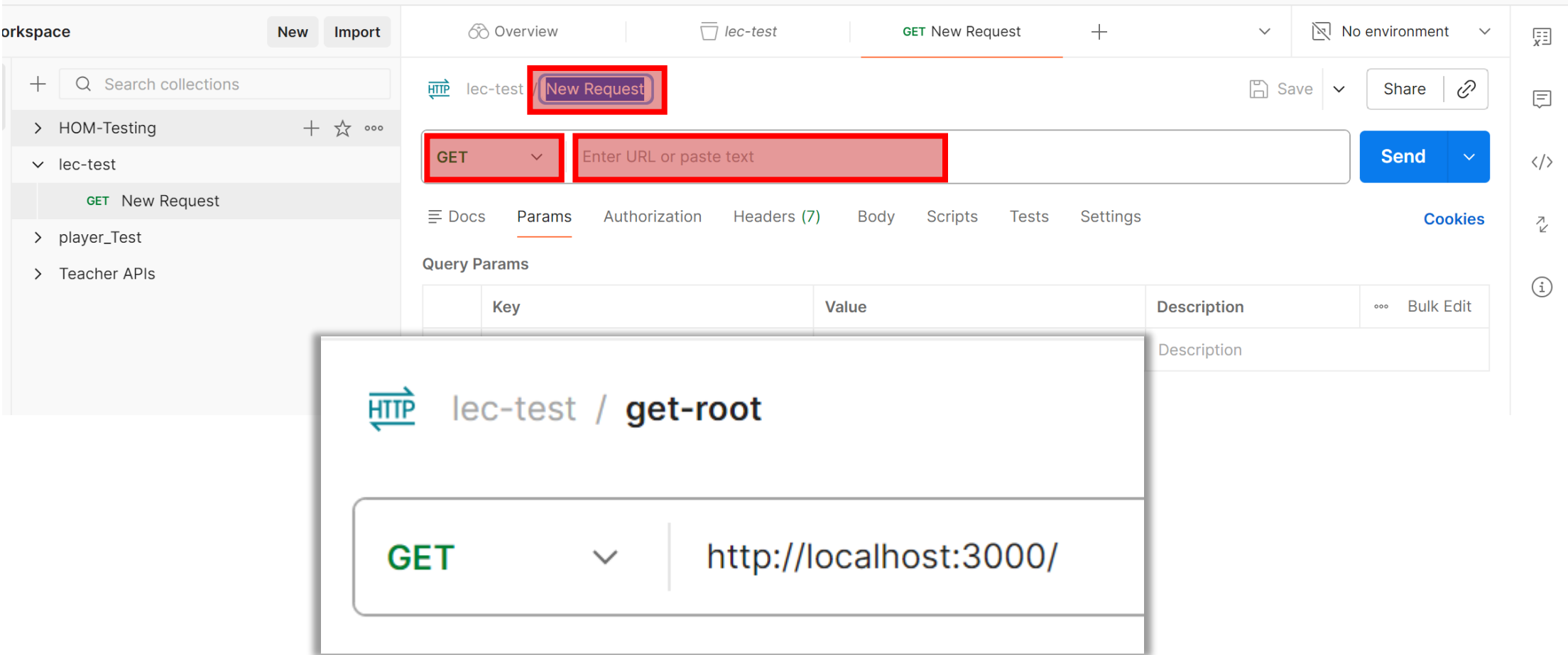
Overview Authorization



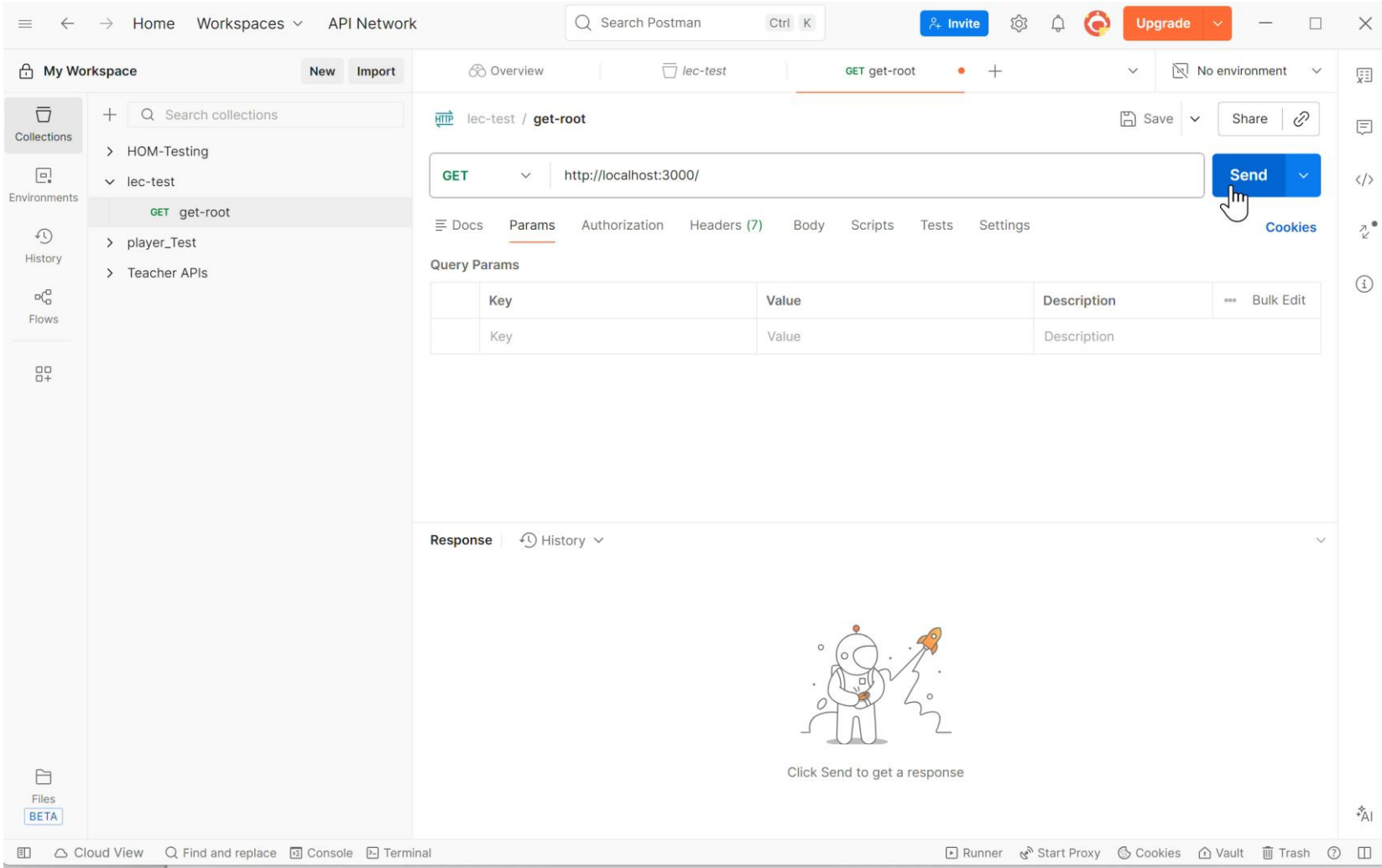
Make things easier for your teammates with a complete

[View complete documentation →](#)

Postman Application



Postman Application



Database Connections with Node.JS

- Create a new folder and name it 'dbwebservice'
- Open this folder in VS Code
- Open terminal or command prompt in VS Code
- Run the command

`npm init -yes`

Configure the Project

- In VS Code, open terminal:
 - Terminal -> New Terminal
- Run the following commands in this terminal to install **express** and **mysql** packages

npm i express
npm i mysql2



npm i express mysql2

npm i -D nodemon

MYSQL Documentation: <https://www.npmjs.com/package/mysql>

package.json

Initial file

```
{ package.json •
{} package.json > ...
1 {
2   "name": "code-examples",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "type": "commonjs"
13 }
```

```
1 {
2   "name": "dbwebservice",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "dev": "nodemon index.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "type": "commonjs",
13  "dependencies": {
14    "express": "^5.2.1",
15    "mysql2": "^3.20.0"
16  },
17  "devDependencies": {
18    "nodemon": "^3.1.14"
19  }
20 }
```

Database in XAMPP

- Open XAMPP
- Open Admin for MySQL
- Open **nodejsdb** that we have created earlier
- Import file **students-table.sql** file (download it from LMS or GitHub)
 - It creates a students table and add 30 records

Import SQL File

localhost/phpmyadmin/index.php?route=/database/structure&db=nodejsdb

Bookmarks Fun Courses KSA-Banks KAU Jobs Islam Journals Google Translate Suggested Sites References SDN NanoNetworks NZ Life NLP All Bookmarks

phpMyAdmin

Recent Favorites

New

information_schema

mysql

nodejsdb

performance_schema

phpmyadmin

test

Server: 127.0.0.1 » Database: nodejsdb

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Tracking Designer More

No tables found in database.

Create new table

Table name

Number of columns

4

Create

Import SQL File

localhost/phpmyadmin/index.php?route=/database/import&db=nodejsdb

Server: 127.0.0.1 » Database: nodejsdb

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Tracking Designer More

phpMyAdmin

Recent Favorites

- New
- information_schema
- mysql
- nodejsdb
- performance_schema
- phpmyadmin
- test

Importing into the database "nodejsdb"

File to import:

File may be compressed (gzip, bzip2) or uncompressed.
A compressed file's name must end in **.[format].[compression]**. Example: **.sql.zip**

Browse your computer: (Max: 40MiB)

Choose File students-table.sql

You may also drag and drop a file on any page.

Character set of the file:

utf-8

Structure of **students** Table

```
CREATE TABLE IF NOT EXISTS students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    gender VARCHAR(10),  
    semester INT NOT NULL,  
    major ENUM('CS', 'IS', 'SE') NOT NULL,  
    -- Add constraints  
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8)  
);
```

index.js

- Create a new file in the folder and name it index.js
- Import express and mysql packages

```
const express = require("express");  
const mysql = require("mysql2");  
const app = express();
```


Db Connection

- Create the database connection using **mysql**

```
const dbCon = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "",  
  database: "nodejsdb",  
});
```

Db Connection

- Check the database connection

```
dbCon.connect((error) => {  
  if (error) {  
    console.log("Error Db: " + error.message);  
    throw error;  
  } else {  
    console.log("Db is connected successfully...");  
  }  
});
```

'/' Route

Welcome to Database Handling in Node.JS

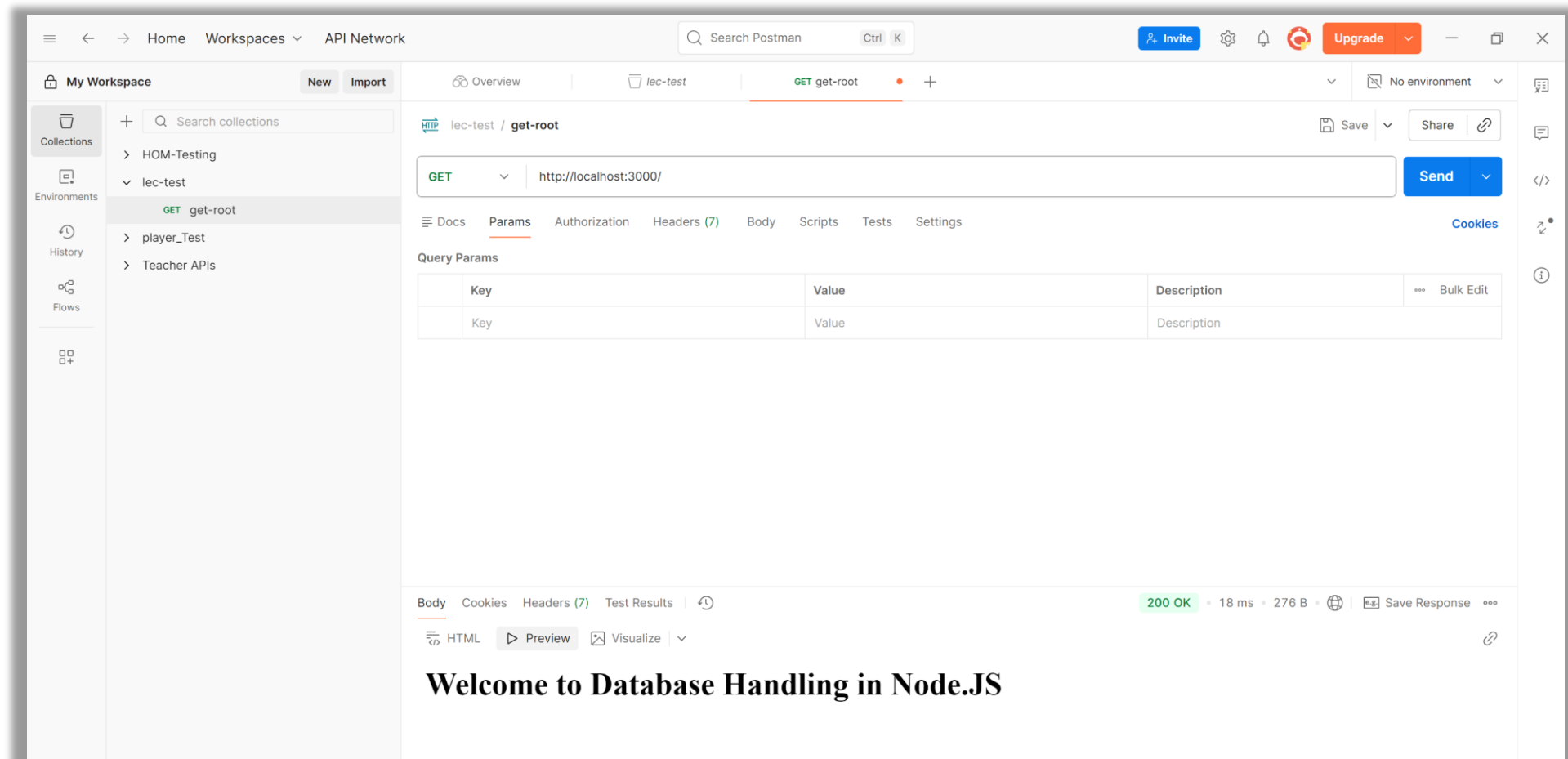
- Let us implement the root route for this app

```
app.get("/", (req, res) => {
  res.send("<h1>Welcome to Database Handling in Node.JS</h1>");
});
```

- Start to listen HTTP request on Server

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Listening on port
${PORT}...`));
```

'/' Route



Run the App

```
malik@DESKTOP-HVT445C MINGW64 /d/Teaching/IS311 Web Development/Lectures/Week 09/code252/dbwebservice
○ $ npm run dev

> dbwebservice@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Listening on port 3000...
Db is connected successfully...
█
```

GET '/data/students' Route

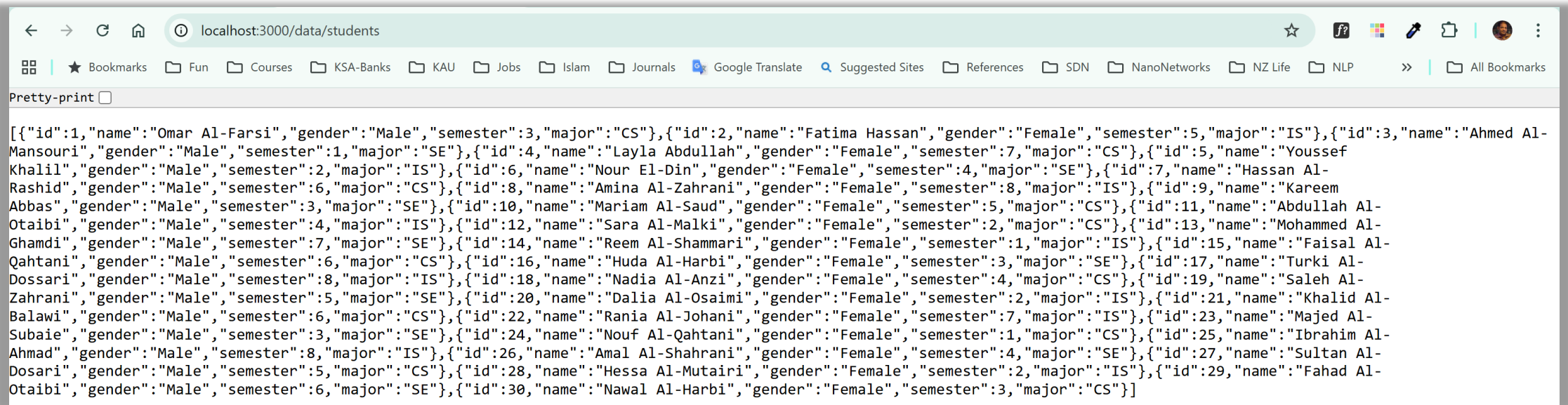
```
app.get("/data/students", (req, res) => {  
  // writing query to execute in MySQL  
  let sqlQuery = "SELECT * FROM students";  
  // executing the query in MySQL and getting back results  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) throw error;  
    // sending back the records to client  
    res.send(results);  
  });  
});
```

GET '/data/students' Route Flow

- Browser (Client) send **Get** request on **localhost:3000/data/students** request to server - index.js file running via nodemon
- Code will accept this request
- Send query to DB Server
- Receive error or data
- Send back records to browser (Client)

```
app.get("/data/students", (req, res) => {  
  // writing query to execute in MySQL  
  let sqlQuery = "SELECT * FROM students";  
  // executing the query in MySQL and getting back results  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) throw error;  
    // sending back the records to client  
    res.send(results);  
  });  
});
```

GET '/data/students' Route



```
[{"id":1,"name":"Omar Al-Farsi","gender":"Male","semester":3,"major":"CS"}, {"id":2,"name":"Fatima Hassan","gender":"Female","semester":5,"major":"IS"}, {"id":3,"name":"Ahmed Al-Mansouri","gender":"Male","semester":1,"major":"SE"}, {"id":4,"name":"Layla Abdullah","gender":"Female","semester":7,"major":"CS"}, {"id":5,"name":"Youssef Khalil","gender":"Male","semester":2,"major":"IS"}, {"id":6,"name":"Nour El-Din","gender":"Female","semester":4,"major":"SE"}, {"id":7,"name":"Hassan Al-Rashid","gender":"Male","semester":6,"major":"CS"}, {"id":8,"name":"Amina Al-Zahrani","gender":"Female","semester":8,"major":"IS"}, {"id":9,"name":"Kareem Abbas","gender":"Male","semester":3,"major":"SE"}, {"id":10,"name":"Mariam Al-Saud","gender":"Female","semester":5,"major":"CS"}, {"id":11,"name":"Abdullah Al-Otaibi","gender":"Male","semester":4,"major":"IS"}, {"id":12,"name":"Sara Al-Malki","gender":"Female","semester":2,"major":"CS"}, {"id":13,"name":"Mohammed Al-Ghamdi","gender":"Male","semester":7,"major":"SE"}, {"id":14,"name":"Reem Al-Shammari","gender":"Female","semester":1,"major":"IS"}, {"id":15,"name":"Faisal Al-Qahtani","gender":"Male","semester":6,"major":"CS"}, {"id":16,"name":"Huda Al-Harbi","gender":"Female","semester":3,"major":"SE"}, {"id":17,"name":"Turki Al-Dossari","gender":"Male","semester":8,"major":"IS"}, {"id":18,"name":"Nadia Al-Anzi","gender":"Female","semester":4,"major":"CS"}, {"id":19,"name":"Saleh Al-Zahrani","gender":"Male","semester":5,"major":"SE"}, {"id":20,"name":"Dalia Al-Osaimi","gender":"Female","semester":2,"major":"IS"}, {"id":21,"name":"Khalid Al-Balawi","gender":"Male","semester":6,"major":"CS"}, {"id":22,"name":"Rania Al-Johani","gender":"Female","semester":7,"major":"IS"}, {"id":23,"name":"Majed Al-Subaie","gender":"Male","semester":3,"major":"SE"}, {"id":24,"name":"Nouf Al-Qahtani","gender":"Female","semester":1,"major":"CS"}, {"id":25,"name":"Ibrahim Al-Ahmad","gender":"Male","semester":8,"major":"IS"}, {"id":26,"name":"Amal Al-Shahrani","gender":"Female","semester":4,"major":"SE"}, {"id":27,"name":"Sultan Al-Dosari","gender":"Male","semester":5,"major":"CS"}, {"id":28,"name":"Hessa Al-Mutairi","gender":"Female","semester":2,"major":"IS"}, {"id":29,"name":"Fahad Al-Otaibi","gender":"Male","semester":6,"major":"SE"}, {"id":30,"name":"Nawal Al-Harbi","gender":"Female","semester":3,"major":"CS"}]
```


GET '/data/students' Route

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/data/students`. The response is a 200 OK status with a 23 ms response time and 2.51 KB of data. The response body is a JSON array of four student objects.

Key	Value	Description
Key	Value	Description

```

1  [
2    {
3      "id": 1,
4      "name": "Omar Al-Farsi",
5      "gender": "Male",
6      "semester": 3,
7      "major": "CS"
8    },
9    {
10     "id": 2,
11     "name": "Fatima Hassan",
12     "gender": "Female",
13     "semester": 5,
14     "major": "IS"
15   },
16   {
17     "id": 3,
18     "name": "Ahmed Al-Mansouzi",
19     "gender": "Male",
20     "semester": 1,
21     "major": "SE"
22   },
23   {
24     "id": 4,
25     "name": "
  
```

POST '/data/student' Route

```
app.post("/data/student", (req, res) => {
  const student = {
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `INSERT INTO students (name, gender, semester, major) VALUES
    ('${student.name}', '${student.gender}', ${student.semester}, '${student.major}')`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is added.</h2>");
  });
});

// setting up app to receive JSON in request body
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

POST '/data/student' Route

The screenshot shows the Postman interface for a REST client. On the left, the 'My Workspace' sidebar displays a collection named 'lec-test' containing three requests: 'get-root', 'get-all-records', and 'add-record'. The 'add-record' request is selected and highlighted. The main panel shows the details of the 'POST' request to 'http://localhost:3000/data/student'. The 'Body' tab is active, showing a JSON payload:

```
{  "name": "Abbas Malik",  "gender": "Male",  "semester": "8",  "major": "CS"}
```

. The 'raw' radio button is selected, and the 'JSON' format is chosen. The 'Send' button is visible. At the bottom, the response status is '200 OK' with a response time of 45 ms and a size of 257 B. The 'Body' tab is also active in the response section, showing the same JSON payload.

POST `http://localhost:3000/data/student` **Send**

Docs Params Authorization Headers (9) **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```

1  {
2      "name": "Abbas Malik",
3      "gender": "Male",
4      "semester": "8",
5      "major": "CS"
6  }
7

```

Body Cookies Headers (7) Test Results | 200 OK • 45 ms • 257 B • Save Response

HTML Preview Visualize

One record is added.

GET '/data/student' Route

- Read only one record from Database Table

```
app.get("/data/student", (req, res) => {  
  let sqlQuery = `SELECT * FROM students WHERE id=${req.body.id}`;  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) throw error;  
    res.send(results);  
  });  
});
```

GET '/data/student' Route

The screenshot displays the Postman interface with a workspace named 'My Workspace'. The left sidebar shows a collection named 'lec-test' containing several API requests. The main panel shows the 'get-one-record' request, which is a GET request to the URL 'http://localhost:3000/data/student'. The request is configured with the 'raw' body type and 'JSON' format. The response is a 200 OK status with a response time of 75 ms and a body size of 313 B. The response body is a JSON object containing student information.

Request:

```
GET http://localhost:3000/data/student
```

Response:

```
{
  "id": 16,
  "name": "Huda Al-Harbi",
  "gender": "Female",
  "semester": 3,
  "major": "SE"
}
```

GET '/data/student/id' Route

- Read only one record - id as param

```
app.get("/data/student", (req, res) => {  
  let sqlQuery = `SELECT * FROM students WHERE id=${req.params.id}`;  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) throw error;  
    res.send(results);  
  });  
});
```

GET '/data/student/id' Route

The screenshot shows the Postman interface with a workspace named 'My Workspace'. The left sidebar displays a collection named 'lec-test' under 'HOM-Testing', which contains several API endpoints. The 'get-one-param' endpoint is selected. The main panel shows the request details for a GET request to 'http://localhost:3000/data/student/16'. The 'Body' tab is active, showing 'none' as the request body. The response section at the bottom shows a '200 OK' status with a response time of 40 ms and a size of 313 B. The response body is displayed in JSON format, containing student information for ID 16.

Request:

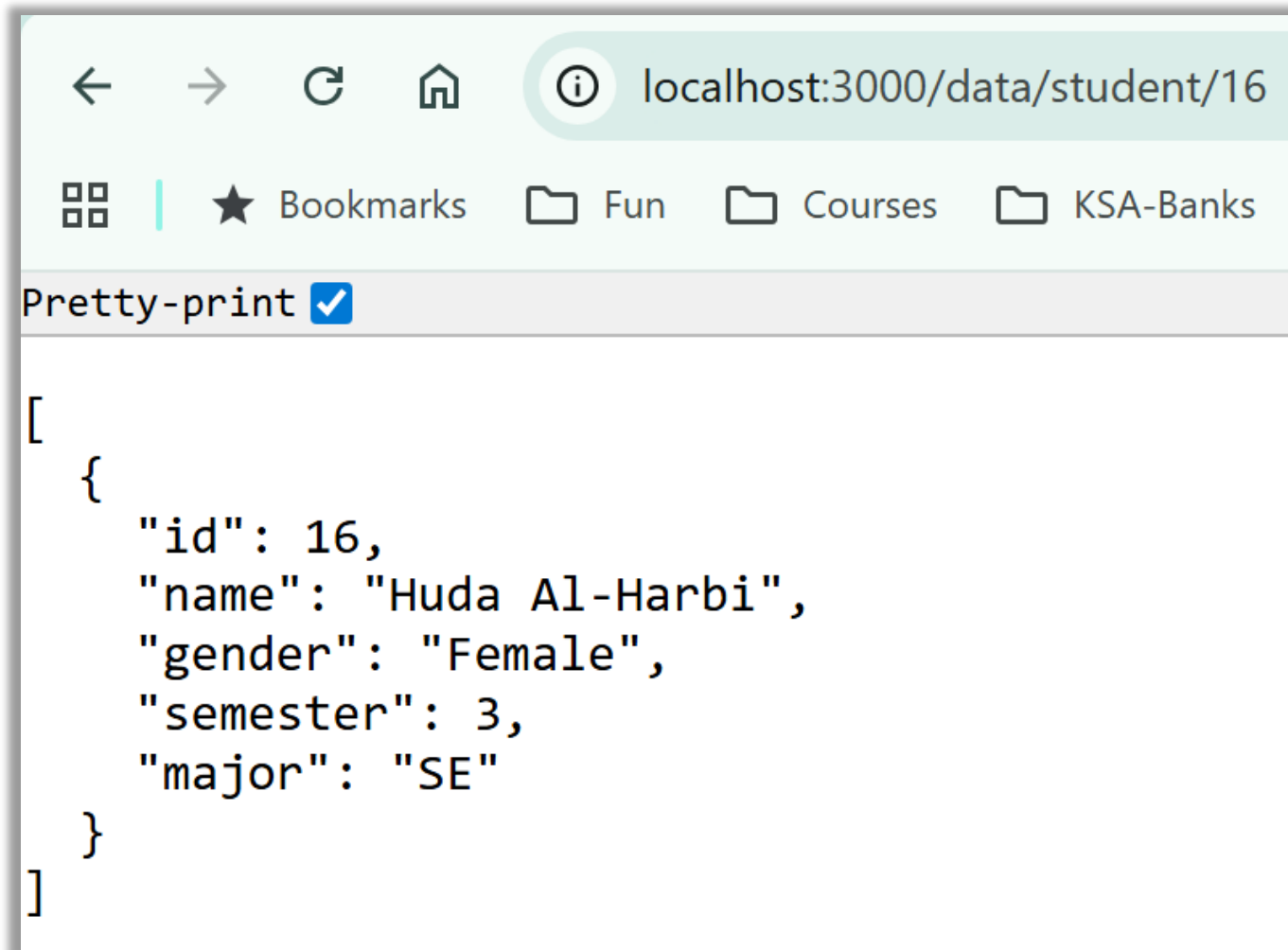
- Method: GET
- URL: http://localhost:3000/data/student/16
- Body: none

Response:

```

1 [
2   {
3     "id": 16,
4     "name": "Huda Al-Harbi",
5     "gender": "Female",
6     "semester": 3,
7     "major": "SE"
8   }
9 ]
  
```

GET '/data/student/id' Route



A screenshot of a web browser window. The address bar shows 'localhost:3000/data/student/16'. Below the address bar, there are bookmarks for 'Bookmarks', 'Fun', 'Courses', and 'KSA-Banks'. A 'Pretty-print' checkbox is checked. The main content area displays a JSON array containing a single object with the following properties: 'id' (16), 'name' ('Huda Al-Harbi'), 'gender' ('Female'), 'semester' (3), and 'major' ('SE').

```
[
  {
    "id": 16,
    "name": "Huda Al-Harbi",
    "gender": "Female",
    "semester": 3,
    "major": "SE"
  }
]
```


PUT

'/data/student'

Route

```
app.put("/data/student", (req, res) => {
  const student = {
    id: req.body.id,
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `UPDATE students
  Set
    name = '${student.name}',
    gender = '${student.gender}',
    semester = ${student.semester},
    major = '${student.major}'
  WHERE id = ${student.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is updated.</h2>");
  });
});
```



PUT

'/data/student/id'

Route

```
app.put("/data/student/:id", (req, res) => {
  const student = {
    id: req.params.id,
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `UPDATE students
  Set
  name = '${student.name}',
  gender = '${student.gender}',
  semester = ${student.semester},
  major = '${student.major}'
  WHERE id = ${student.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is updated.</h2>");
  });
});
```

PUT '/data/student/id' Route

The screenshot shows a REST client interface with the following details:

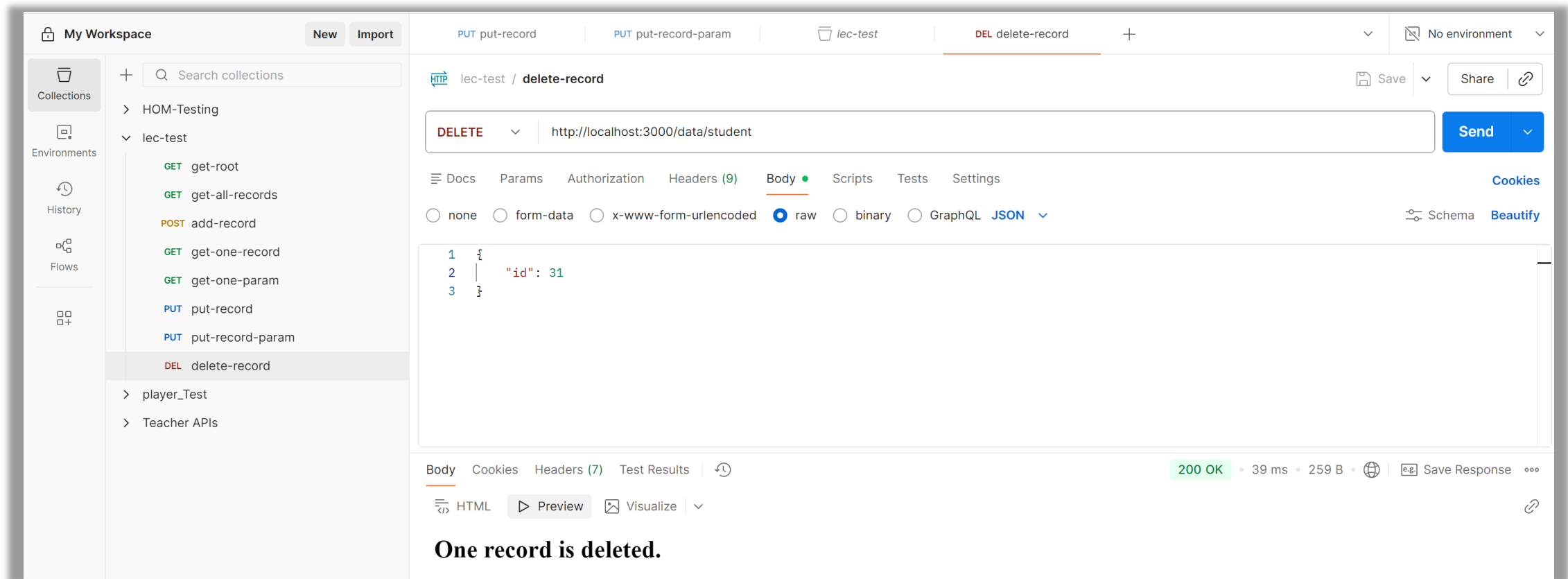
- Workspace:** My Workspace
- Left Panel:**
 - Collections: Search collections
 - Environments: lec-test
 - GET get-root
 - GET get-all-records
 - POST add-record
 - GET get-one-record
 - GET get-one-param
 - PUT put-record
 - PUT put-record-param (selected)
 - History: player_Test, Teacher APIs
 - Flows
- Request Tab:** PUT put-record-param
 - Method: PUT
 - URL: http://localhost:3000/data/student/31
 - Body (raw):


```
1 {
2   "name": "M. G. Abbas Malik",
3   "gender": "Male",
4   "semester": 7,
5   "major": "SE"
6 }
7
```
- Response Tab:**
 - Status: 200 OK
 - Time: 52 ms
 - Size: 259 B
 - Message: One record is updated.

DELETE '/data/student' Route

```
app.delete("/data/student", (req, res) => {  
  let sqlQuery = `DELETE FROM students  
    WHERE id = ${req.body.id}`;  
  dbCon.query(sqlQuery, (error, results) => {  
    if (error) throw error;  
    res.send("<h2>One record is deleted.</h2>");  
  });  
});
```

DELETE '/data/student' Route



The screenshot shows the Postman interface with the following details:

- Workspace:** My Workspace
- Collections:** lec-test (expanded)
- Request:** DELETE http://localhost:3000/data/student
- Body:**

```

1 {
2   "id": 31
3 }

```
- Response:** 200 OK, 39 ms, 259 B. Body: One record is deleted.

DELETE '/data/student/id' Route

```
app.delete("/data/student/:id", (req, res) => {  
  let sqlQuery = `DELETE FROM students  
    WHERE id = ${req.params.id}`;  
  dbCon.query(sqlQuery, (error, results) => {  
    if (error) throw error;  
    res.send("<h2>One record is deleted.</h2>");  
  });  
});
```

DELETE '/data/student/id' Route

My Workspace

New Import

Search collections

HOM-Testing

lec-test

GET get-root

GET get-all-records

POST add-record

GET get-one-record

GET get-one-param

PUT put-record

PUT put-record-param

DEL delete-record

DEL delete-record-param

player_Test

Teacher APIs

PUT put-record

PUT put-record-param

DEL delete-record-param

DEL delete-record

POST add-record

lec-test / delete-record-param

DELETE

http://localhost:3000/data/student/32

Docs

Params

Authorization

Headers (7)

Body

Scripts

Tests

Settings

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (7)

Test Results

200 OK

30 ms

259 B

Save Response

HTML

Preview

Visualize

One record is deleted.

HTTP Request Status

[Complete List](#)

- HTTP response status codes indicate whether a specific [HTTP](#) request has been successfully completed. Responses are grouped in five classes:
 1. [Informational responses](#) (100 - 199)
 2. [Successful responses](#) (200 - 299)
 3. [Redirection messages](#) (300 - 399)
 4. [Client error responses](#) (400 - 499)
 5. [Server error responses](#) (500 - 599)

Set HTTP Request Status

```
app.get("/data/students", (req, res) => {  
  // writing query to execute in MySQL  
  let sqlQuery = "SELECT * FROM students";  
  // executing the query in MySQL and getting back results  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) throw error;  
    // sending back the records to client  
    res.status(200).send(results);  
  });  
});
```

```
res.status(200);  
res.send(results);
```

Set HTTP Request Status

```
app.get("/data/student", (req, res) => {  
  let sqlQuery = `SELECT * FROM students WHERE id=${req.body.id}`;  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) {  
      res.status(500).send({ message: "Internal Server Error.." });  
    }  
    res.status(200).send({ success: true, data: results[0] });  
  });  
});
```

HTTP Request Labwork

- Create GET, PUT, POST and DELETE for Course with validations
 - ID is a number, autogenerated and primary key,
 - Name is required, string and at least 5 character long.
 - Semester is between 1 and 8 only
 - Year can be between 1 to 4 only
 - Major can have values: CS, IS or SE
- Structure of Course is:

```
course {  
  id: Number,  
  name: String,  
  code: String,  
  semester: Number,  
  major: String,  
  year: Number,  
}
```

Data Validation

- We can write a code to validate the data send by the use via HTTP request
- Security best practice is always validate the user data
- Cannot trust the user about the data
- Status: 400
 - Means bad request

Data Validation

Getting Started

```
if(!req.body.name || req.body.name.length < 3)  
  return res.status(400)  
    .send('Not a valid data: Name is missing or nor 3 character long!!!');
```

JOI - Node Package that will make data validation easier

```
npm i joi
```

Data Validation with **JOI**

- Import this package in your code

```
const Joi = require('joi');
```

- Build validation rules using **Joi**

```
const nameVlvalidationSchema = Joi.object({  
  name: Joi.string().min(3).required(),  
});
```

Data Validation with JOI

- Validate with Joi

```
const result = nameVlidaionSchema.validate(req.body);
```

- Log this result

```
Listening on port 3000... Success
{ value: { name: 'M. Rizwan' } }
```

```
Listening on port 3000...
{
  value: {},
  error: [Error [ValidationError]: "name" is required] {
    _original: {},
    details: [ [Object] ]
  }
}
```


Data Validation with JOI

- Validation Error with Joi

```
console.log(result.error);
```

```
[Error [ValidationError]: "name" is required] {  
  _original: {},  
  details: [  
    {  
      message: '"name" is required',  
      path: [Array],  
      type: 'any.required',  
      context: [Object]  
    }  
  ]  
}
```

```
console.log(result.error.details[0].message);
```

Multiple Field Validation

```
const studentValidationSchema = Joi.object({  
  name: Joi.string().min(3).max(50).required(),  
  gender: Joi.string().required(),  
  semester: Joi.number().min(1).max(8).required(),  
});
```

```
// validating the student record using JOI  
const { error } = studentValidationSchema.validate(studentInfo);  
  
if (error) {  
  // one or more validation(s) have failed  
  return res.send({ result: "failure", message: error.details[0].message });  
}
```

Multiple Field Validation

localhost:3001/addstudent.html?name=Abbas&gender=male&semester=10

Students Database App

Add a Students in Database

[Go Back to Home](#)

Student Name:

Student Gender:

Student Semester:

localhost:3001 says

Student is not added - "name" length must be at least 3 characters long

Multiple Field Validation

localhost:3001/addstudent.html?name=Ab&gender=male&semester=10

Students Database App

Add a Students in Database

[Go Back to Home](#)

Student Name:

Student Gender:

Student Semester:

localhost:3001 says

Student is not added - "semester" must be less than or equal to 8

[OK](#)

POST with Data Validation

```
const studentValidationSchema = Joi.object({
  name: Joi.string().min(3).max(50).required(),
  gender: Joi.string().required(),
  semester: Joi.number().min(1).max(8).required(),
});
```

```
app.post("/data/students", (req, res) => {
  const studentInfo = req.body;
  if (!studentInfo)
    return res.send({ result: "failure", message: "Body is missing" });
  if (!studentInfo.name || !studentInfo.gender || !studentInfo.semester)
    return res.send({
      result: "failure",
      message: "Body does not contain student info",
    });
});
```

```
// validating the student record using JOI
const { error } = studentValidationSchema.validate(studentInfo);
```

```
if (error) {
  // one or more validation(s) have failed
  return res.send({ result: "failure", message: error.details[0].message });
}
```

```
try {
  let sqlQuery = `INSERT INTO students(name, gender, semester)
  VALUES("${studentInfo.name}", "${studentInfo.gender}", ${studentInfo.semester})`;

  dbConn.query(sqlQuery, (error, result) => {
    if (error) throw error;
  });
}
```

Email and Confirm Email Validation

Email Verification

Email address

Confirm Email address

Email and ConfirmEmail Validation

```
<body>
  <div class="container">
    <div class="email-form">
      <h2 class="form-title">Email Verification</h2>
      <form id="emailForm" class="needs-validation" novalidate>
        <div class="mb-3">
          <label for="email" class="form-label">Email address</label>
          <input type="email" class="form-control" id="email" required
            placeholder="Enter your email"/>
        </div>
        <div class="mb-3">
          <label for="confirmEmail" class="form-label">Confirm Email address</label>
          <input type="email" class="form-control" id="confirmEmail" required
            placeholder="Re-enter your email"
          />
        </div>
        <button type="submit" class="btn btn-primary submit-btn">
          Submit
        </button>
      </form>
    </div>
  </div>
</body>
```

Email and ConfirmEmail Validation

```
const emailValidationSchema = Joi.object({  
  email: Joi.string().min(8).max(16).required(),  
  confirmEmail: Joi.ref("email"),  
});
```

- confirmEmail must be present and equal to email.
- Validate that two fields are same

Learn more about JOI and Validation

- [Tutorial 1](#)
- [Tutorial 2](#)

Reference

- <https://www.npmjs.com/package/mysql>
- https://www.w3schools.com/nodejs/nodejs_mysql.asp
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>